CSE 151B/251B Milestone Report

Qingyuan Yu
Department of Mathematics
University of California San Diego
qiy005@ucsd.edu

Wangji Zhang
Department of Mathematics
University of California San Diego
chz091@ucsd.edu

Po-Chen Lin
Halıcıoğlu Data Science Institute
University of California San Diego
pochen8787@gmail.com

Abstract

In this project, we explore the capability of a small language model, Qwen3-4B-Thinking-2507, to solve mathematical problems across various domains and difficulty levels. Up to this milestone, we have focused on prompt engineering, asking the model to output in strict formats for multiple-choice answers, multi-part free-form answers, numerical precision, and final boxed outputs. We evaluate the model using answer-level accuracy: for multiple-choice questions, we extract the predicted letter from the response and compare it to the given answer; for free-form questions, we use the provided `Judger` class to compare the generated response against the gold answer. The starter baseline reached a public Kaggle score of 0.575, while our prompt-engineered configuration raises validation accuracy from 56.44% to 62.67% (+6.23 pp) on a stratified 225-question split, with the largest gain (+13.33 pp) on multiple-choice and a drop in the rate of unparsable responses from 17.4% to 3.6%. The corresponding Kaggle submission is in flight; the leaderboard score will be reported in the final paper.

1 Introduction

Problem Definition. Our task is to build a small LLM system that can run locally on a consumer-grade GPU and answer mathematical reasoning questions. The question styles include multiple-choice and free-form questions; free-form questions may require one or more numerical or symbolic answers, often corresponding to `[ANS]` placeholders in the problem statement. The grader extracts the contents of the final boxed expression $\boxed{\cdot}$ and judges semantic equivalence with the gold answer using a SymPy-based normalizer. We aim to improve the model's answer-level accuracy because the starter system performs poorly, achieving a public Kaggle score of 0.575. At this milestone our approach is prompt engineering rather than model retraining: we give the model precise, type-specific format instructions so that correct reasoning is no longer wasted on unparsable output.

Problem Significance. Modern LLMs such as ChatGPT and Claude can solve most of these questions correctly, but they require large amounts of compute, an internet connection, and recurring API fees, which makes them unsuitable for on-device tutoring, private-data settings, or reproducible academic baselines. A compact reasoning model that runs on a single 24 GB consumer GPU is the natural alternative, but it exposes practical issues that

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

frontier models hide: quantization-induced regressions, KV-cache pressure from long chains of thought, and the gap between a correct answer and a correctly formatted one.

Technical Challenge. The task is hard along five orthogonal axes. (1) Data heterogeneity. Three answer formats coexist; multi-answer questions are graded all-or-nothing. (2) Compute. A 4B-parameter thinking model emits long chain-of-thought traces (delimited by dedicated `<think>` tokens) that easily exceed 10^4 tokens; with 24 GB of VRAM, INT4 quantization is mandatory to fit batched inference. (3) Software stack. The latest `transformers` (5.x) ships breaking changes that the latest released vLLM (0.20.0) has not yet adapted; a working version pinning must be discovered by hand. (4) Format strictness. The grader silently rejects `\boxed{(C)}` or `\boxed{C.}` even when the underlying letter is correct. (5) Truncation. Long chains of thought can hit `max_tokens` before the model emits a final boxed answer, costing the entire question.

Contributions. The starter notebook supplies a working but slow inference path (HuggingFace `transformers` with BitsAndBytes, $\approx$ 30 h estimated for a single full pass) and a generic prompt that produces format failures on 17.4% of public responses. Our key contributions in this milestone are:

- A reproducible vLLM-based inference pipeline that runs the full 1,126 public set in roughly 3 hours on a single RTX 4090, with a previously undocumented network configuration fix (forcing the inter-process socket to loopback) and a verified version pinning (`transformers` 4.57.6, vLLM 0.20.0, INT4 BitsAndBytes).
- A taxonomy of failure modes derived from a real submission, separating format failures (truncated, missing boxed answer, multi-part shape mismatch) from genuine reasoning errors, which we use as the design target for the prompt rewrite.
- A format-aware prompt rewrite, validated on a stratified 20% split, that improves overall accuracy by +6.23 pp on 225 held-out questions and reduces the rate of responses without a final boxed answer from 17.4% to 3.6%.
- An open-source modular pipeline (`src/cse151b_comp/{inference, extract, evaluate, error_analysis, submission, eval_harness}`) with 85 unit tests, designed so Phase 2 (self-consistency) and Phase 4 (QLoRA fine-tuning) plug in without further refactoring.

2 Related Work

Chain-of-thought reasoning. Wei et al. [8] showed that prompting language models to produce intermediate reasoning steps before a final answer improves performance on arithmetic, commonsense, and symbolic tasks. Qwen3-Thinking [6] is post-trained to emit such traces inside dedicated thinking tokens and specifically tuned for math and code reasoning, which is what makes a 4B-parameter model competitive on a benchmark like the one in this competition.

Self-consistency. Wang et al. [7] demonstrated that sampling multiple reasoning paths and taking a majority vote over the final answers substantially improves chain-of-thought accuracy. We plan to deploy this technique in Phase 2, with a per-question-type voting strategy that respects multi-answer tuple equality.

Efficient inference and parameter-efficient fine-tuning. vLLM [4] introduces PagedAttention, which we rely on to fit batched generation with long contexts on a single 24 GB GPU. For the planned fine-tuning phase we will use QLoRA [1] on top of the LoRA adapter formulation [3], training on NuminaMath-CoT [5] with explicit decontamination against the public competition data. The MATH benchmark [2] is the historical reference point against which all of these techniques were originally evaluated.

3 Methods

Problem Setting. Let $x_i = (q_i, o_i)$ denote question i, where q_i is the natural-language statement and o_i is a (possibly empty) list of multiple-choice options. The model produces a response $r_i = M(\text{prompt}(x_i))$, from which an extractor $E(r_i)$ returns the contents of the final boxed expression. The grader $g(\hat{y}_i, y_i^*) \in \{0, 1\}$ judges symbolic equivalence with the gold answer y_i^* at 1e$-$8 relative tolerance. We do not train the model in this milestone; instead we optimize over the prompt template and decoding hyperparameters with respect to overall accuracy

$$\text{Acc} \;=\; \frac{1}{N} \sum_{i=1}^{N} g(E(r_i),\, y_i^*)\,.$$

For multi-answer free-form questions, g requires tuple equality after element-wise normalization: any single sub-answer being wrong yields zero credit on the whole problem.

Idea Summary. Our approach rests on the observation that, before any improvement in mathematical capability is possible, a substantial fraction of the starter pipeline's errors are caused by format mismatches that the grader's strict tolerance amplifies into hard failures. We therefore proceed in three steps. First, we stand up a fast, reliable vLLM inference pipeline so that iteration time goes from hours to tens of minutes. Second, we run a single pass on the public set to obtain a baseline submission and a corpus of failure cases. Third, we analyze that corpus to extract a small set of format-aware prompt rules (a multi-answer template, an anti-rounding rule, an MCQ negative-example clause, and a token-budget self-rescue rule), validate the resulting prompt on a stratified validation split, and quantify which question types and length buckets benefit most. The risk of overfitting prompts to validation is mitigated by stratifying by question type and freezing the validation indices with a fixed seed at the start of the project.

4 Experiments

4.1 Baselines

We compare two configurations of the same base model on the same data:

- Phase 0 (starter prompt). The unmodified starter prompts ("Solve step-by-step. Put final answer in `\boxed{}`." for free-form; "Output ONLY the letter inside `\boxed{}`." for multiple choice), with `max_tokens` = 12,288 and `temperature` = 0.6. This baseline scores 0.575 on the public Kaggle leaderboard.
- Phase 1 (our prompt). A rewritten system prompt that adds explicit format negative examples, a multi-answer formatting rule, an anti-rounding rule, and a token-budget self-rescue rule (see `src/cse151b_comp/prompts.py`). We also raise `max_tokens` to 16,384 with `max_model_len` bumped to 20,480 to leave 4K tokens of prompt headroom.

4.2 Evaluation

The Kaggle leaderboard scores submissions on 943 private problems and is the ultimate metric. For local development we use the 1,126 public problems with their gold answers, scored by the course-provided `Judger`, a SymPy-based normalizer that handles LaTeX commands, units, and numerical equivalence. For prompt-engineering A/B tests we use a fixed stratified 20% validation set of 225 problems (`data/val_indices.json`, seed = 42), preserving the question-type ratio of the full set (75 MCQ, 67 free-form single, 83 free-form multi). The remaining 80% is reserved for Phase 4 fine-tuning, with a further 10% held out as a clean test harness.

4.3 Implementation Details

All experiments run on a single NVIDIA RTX 4090 (24 GB) provided by Prof. Xiaolong Wang's lab. Software stack: Qwen3-4B-Thinking-2507, vLLM 0.20.0, `transformers` 4.57.6,

`bitsandbytes` 0.49 (INT4 with `bnb_4bit_use_double_quant=True` and BF16 compute), and `torch` 2.11. Decoding follows the official Qwen3-Thinking recommendation: `temperature` = 0.6, `top_p` = 0.95, `top_k` = 20. vLLM parameters: `gpu_memory_utilization` = 0.70, `max_num_seqs` = 64, `max_num_batched_tokens` = 20,480. We do not fine-tune in this milestone, so there is no optimizer or learning rate to tune; the only hyperparameters we sweep are the prompt content and `max_tokens`. A full single-pass inference over 1,126 public questions takes roughly 3 hours wall-clock end-to-end. Crucial workaround. The default vLLM V1 engine binds its inter-process socket to the host's primary network IP, which on this machine resolves to a 100.80.x.x subnet that the OS firewall blocks; this manifested as repeated socket timeouts that never recovered. Setting the environment variable `VLLM_HOST_IP=127.0.0.1` before importing vLLM forces loopback and resolves the issue.

4.4 Results

Headline accuracy. On the stratified validation set, our Phase 1 prompts improve overall accuracy from 56.44% (127/225) to 62.67% (141/225), a +6.23 percentage-point gain. Figure 1 breaks the gain down by question type: the largest improvement is on multiple choice (+13.33 pp), where format-related failures of the starter (e.g., `\boxed{(C)}` or `\boxed{C.}`) caused silent rejections that our explicit negative-example prompt eliminates.

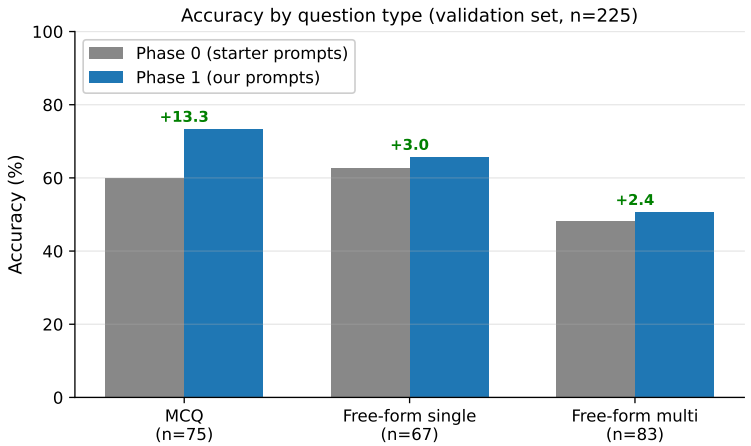

Figure 1: Validation accuracy by question type (n = 225). Numbers above each bar pair are the absolute change in percentage points (Phase 1 minus Phase 0).

Long questions remain a bottleneck. Figure 2 shows that accuracy degrades sharply as the question grows longer. Phase 1 partially closes this gap by raising `max_tokens` and adding a token-budget self-rescue instruction, but very long questions ($\geq$ 1,500 characters) remain at 25% accuracy. We attribute the residual gap to questions whose reasoning genuinely cannot be compressed into our token budget, which Phase 2 self-consistency cannot directly fix; QLoRA on long high-quality CoT traces is a more promising path.

Direct comparison on identical questions. Figure 3 shows the question-by-question outcome matrix on the validation set. Phase 1 picks up 22 new correct answers while regressing on 8, for a net gain of 14 questions—matching the macro +6.23 pp number to within rounding.

Topic-wise behavior. Figure 4 shows accuracy on a regex-tagged topic breakdown. The model is strong on calculus and weak on probability and linear algebra; Phase 1's gains are concentrated on series (+23.1 pp) and calculus (+10.0 pp), with no measurable change on probability or linear algebra. We caution that the topic tagger is a 7-pattern heuristic over the question text and that some topic buckets are small, so we treat these numbers as directional rather than authoritative.

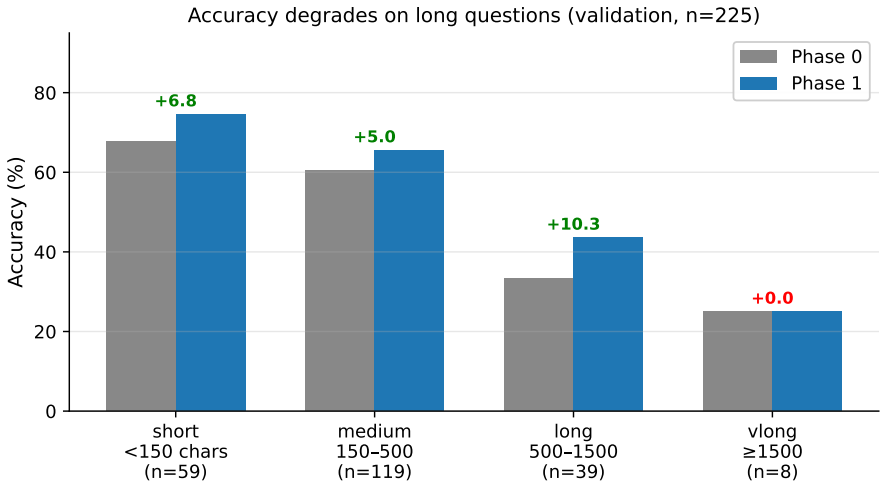

Figure 2: Validation accuracy by question length bucket. Numbers above the bars are the change in percentage points; bucket sizes (n) are shown under each tick. Long and very-long questions are systematically harder.

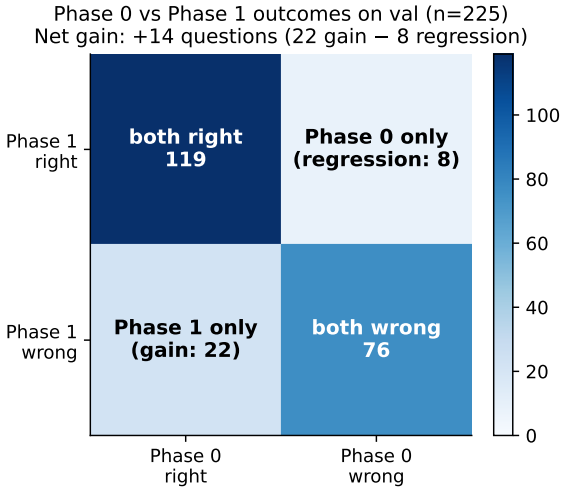

Figure 3: Outcome of Phase 0 vs. Phase 1 on the same 225 validation questions. The off-diagonal cells decompose the gain into wins (22) and regressions (8).

Failure modes on the full public set. The richest signal for what to do next comes from analyzing the 449 incorrect Phase 1 responses on the full 1,126-question public set (Figure 5). 75.7% are responses that emit a final boxed expression with mathematically wrong content—genuine reasoning errors that no prompt rewrite can address. Multi-part shape mismatches account for 15.6% and are the second-largest target; truncation accounts for 7.8%, down from ∼17% once we raised `max_tokens`; the missing-box rate is now 0.9%.

5 Discussion

Achievements. We have raised validation accuracy by +6.23 pp (56.44% → 62.67%), reduced the no-box failure rate by 13.8 pp absolute (17.4% → 3.6%), and cut single-pass inference time on 1,126 questions from an estimated 30 h to ∼3 h. We additionally have a documented taxonomy of failure modes from a real submission, a modular codebase that the next phases plug into without further refactoring, and 85 unit tests guarding each invariant we have committed to (e.g., the multi-part formatting rule and the anti-rounding rule).

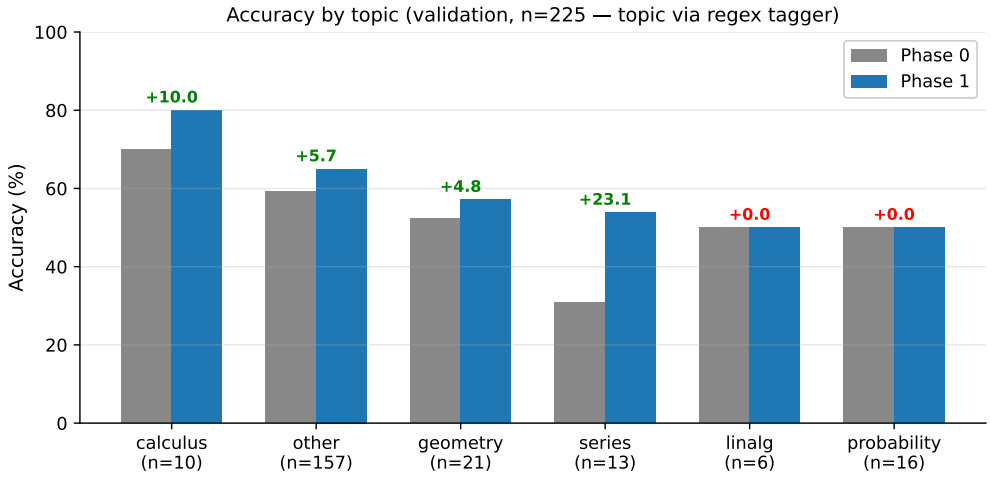

Figure 4: Validation accuracy by regex-tagged topic. Some buckets are small (e.g., $n = 6$ for linear algebra), so absolute numbers should be read alongside the bucket size.

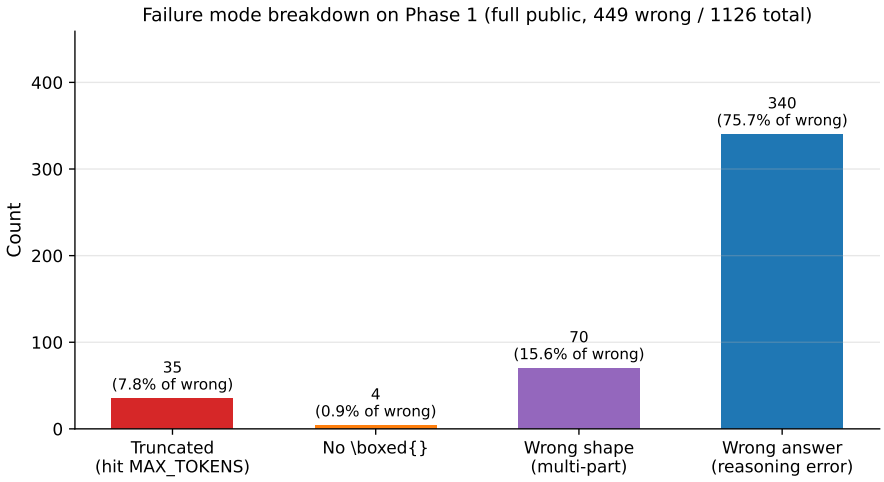

Figure 5: Failure-mode breakdown of the 449 incorrect Phase 1 responses on the full public set ($n = 1{,}126$). The dominant bucket is genuine reasoning errors, which motivates self-consistency and fine-tuning rather than further prompt rewriting.

Bottlenecks and Mitigation. The largest bottleneck during this milestone was the vLLM/`transformers` version mismatch and the network binding bug, which together cost roughly a day; both are now documented in `PLAN.md`. The second-largest bottleneck was that progress bars on Jupyter clients are not robust to client disconnections; we mitigated this by adding a CLI entry point (`cse151b-infer`) that runs detached from any kernel and checkpoints results to disk on completion. The remaining algorithmic bottleneck is the 75.7% of errors that are real reasoning failures—the target of Phase 2 and Phase 4.

Plans Forward. Phase 2 (1–2 days). Self-consistency with $K = 8$ samples at `temperature` $= 0.7$ and a per-question-type voting strategy: letter mode for multiple choice, normalized answer mode for free-form single, and tuple equality for free-form multi. We will run the $K \in \{1, 4, 8\}$ ablation on the same 225 validation set used here and report a "solvable but missed" metric (the number of questions where at least one of the K samples was correct but the vote selected a wrong answer) to upper-bound what better voting alone can achieve. Phase 4 (2–3 days, conditional on Phase 2). Prepare a 50K-example QLoRA training set from NuminaMath-CoT [5] with 8-gram decontamination against `public.jsonl`, train an

r=16 adapter for 2–3 epochs, and ship the LoRA only if a previously unseen 10% holdout improves by ≥ 3 pp over the best non-finetuned configuration.

Team Responsibilities. Qingyuan Yu (Mathematics) led the inference infrastructure and prompt engineering work: the vLLM pipeline, version pinning, the loopback-IP workaround, the four format-aware prompt rules, the modular Python package, the unit-test suite, and the analysis scripts and figures used in this report. Wangji Zhang (Mathematics) led the evaluation and dataset work: the stratified validation split, the SymPy-based `Judger` integration, the failure-mode taxonomy, the `evaluate` and `eval_harness` modules, and the question-type / length / topic breakdown of the validation results. Po-Chen Lin (HDSI) led the literature review and Phase 2/4 design: surveying chain-of-thought, self-consistency, and QLoRA; designing the NuminaMath-CoT decontamination protocol; drafting the self-consistency voting algorithm; and co-writing the report.

Timelines.

- Week 5 (this milestone): vLLM stack, Phase 0 baseline submission, Phase 1 prompt fixes, error analysis, this report.
- Week 6: Phase 2a — self-consistency K-ablation, $K \in \{1, 4, 8\}$.
- Week 7: Phase 2b — NuminaMath-CoT preparation with decontamination, 5K-example pilot fine-tune.
- Week 8: Phase 4 — 50K-example QLoRA fine-tune, first multi-improvement Kaggle submission.
- Week 9: Final ablations, regression-free re-submission, final report writing.
- Week 10: Submission and presentation preparation.

LLM Usage

We used Anthropic's Claude as a research-and-discussion assistant during this milestone in two specific ways: (i) as a literature-search aid to locate canonical papers on chain-of-thought, self-consistency, QLoRA, and vLLM, all of which we read directly and cite from the primary sources; and (ii) as a brainstorming partner to enumerate candidate prompt-engineering interventions and the experimental design needed to validate each one. All code, prompts, experiments, figures, and prose in this report were written and verified by the listed authors, who take full responsibility for any factual or computational inaccuracies. Claude was not used to generate model responses on the competition dataset; the only model evaluated on competition data is Qwen3-4B-Thinking-2507 as required by the rules.

Acknowledgements

We thank Professor Xiaolong Wang for providing access to the GPU compute used to run all experiments reported in this paper. We also thank the course staff for the well-designed starter notebook and grading infrastructure that made the format-failure analysis in Section 4.4 possible.

References

[1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. Advances in Neural Information Processing Systems (NeurIPS), 2023.

[2] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2021.

[3] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In International Conference on Learning Representations (ICLR), 2022.

[4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. ACM Symposium on Operating Systems Principles (SOSP), 2023.

[5] Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath. https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf, 2024.

[6] Qwen Team. Qwen3: Technical report. arXiv preprint arXiv:2505.09388, 2025.

[7] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In International Conference on Learning Representations (ICLR), 2023.

[8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems (NeurIPS), 2022.